

Python Script: main.py

File Name	main.py
Content Type	Python Source Code

[illegible]

[illegible]

[illegible]

```
    )  
    high_picks = [p for f in qualified for p in f["picks"] if p["confidence"] == "HIGH"]  
    mod_picks = [p for f in qualified for p in f["picks"] if p["confidence"] == "MODERATE"]  
  
    msg = f"\nVENUE PICKS - {date_formatted}\n"  
    msg += f"{league}\n\n"  
  
    for result in qualified:  
        league = result.get("league", "")  
        for pick in result["picks"]:  
            emoji = "\ud83d\ude06" if pick["confidence"] == "HIGH" else "\ud83e\udd7a"  
            msg += f">{emoji} <b>{result['fixture']}</b>\n"  
            if league:  
                msg += f">   {league}\n"  
                msg += f">      {pick['type']}\n"  
                msg += f">      {pick['occurrence']} | {pick['confidence']}\n\n"  
  
    msg += f">\n  
\n"  
    msg += f">\n HIGH: {len(high_picks)} MODERATE: {len(mod_picks)}\n"  
    msg += f">\n Fixtures checked: {len(qualified) + len(skipped)}\n"  
    msg += f">\n VenuePicks Bot"  
  
    return msg  
  
# Main Runner  
  
def run(target_date=None):  
    if target_date is None:  
        target_date = (datetime.utcnow() + timedelta(days=1)).strftime("%Y-%m-%d")  
  
    print(f"\n{'='*50}")  
    print(f"VenuePicks Bot - {target_date}")  
    print(f"\n{'='*50}")  
  
    remaining = check_api_status()  
  
    if remaining < 10:  
        send_telegram(  
            f"><b>VenuePicks Bot</b>\n\n"  
            f"API limit almost reached ({remaining} requests left today).\n"  
            f"Picks will resume tomorrow."  
        )  
        return  
  
    # ONE call to get all fixtures for the date  
    fixtures = get_all_fixtures_for_date(target_date)  
  
    if not fixtures:  
        send_telegram(  
            f"><b>VenuePicks Bot</b>\n\n"  
            f"No fixtures found for {target_date} in monitored leagues.\n"  
            f"May be a rest day."  
        )  
        return  
  
    qualified = []  
    skipped = []  
  
    for fix in fixtures:  
        home_team = fix["teams"]["home"]  
        away_team = fix["teams"]["away"]  
        league_name = fix.get("league", {}).get("name", "")  
        fixture_name = f'{home_team["name"]} vs {away_team["name']}'  
        home_id = home_team["id"]  
        away_id = away_team["id"]  
  
        print(f"[INFO] {fixture_name} ({league_name})")  
  
        h2h = get_h2h(home_id, away_id, last=5)  
        picks, reason = analyze_h2h(fixture_name, h2h)  
  
        if picks:  
            qualified.append({  
                "fixture": fixture_name,  
                "league": league_name,  
                "picks": picks  
            })  
            print(f"> {pick['type']} for {pick['team']}")  
        else:
```

```
        skipped.append({"fixture": fixture_name, "reason": reason})
        print(f"    ■ {reason}")

    print(f"\n[INFO] Qualified: {len(qualified)} | Skipped: {len(skipped)}")
    send_telegram(build_telegram_message(qualified, skipped, target_date))
    print("[INFO] Done.")

if __name__ == "__main__":
    run()
```